

Embedded System

AD25T

Introduction to Embedded Systems

➤ History

Embedded systems have their roots in the 1960s.

The Apollo Guidance Computer (AGC), developed by MIT in 1965, was the first widely recognized embedded system for NASA's Apollo Program. It controlled spacecraft, performed complex calculations, and provided unprecedented accuracy and precision through advanced software and hardware.

Apollo Lunar Module



Apollo Guidance Computer

[The Apollo Guidance Computer \(AGC\)](#)

➤ What Are Embedded Systems?

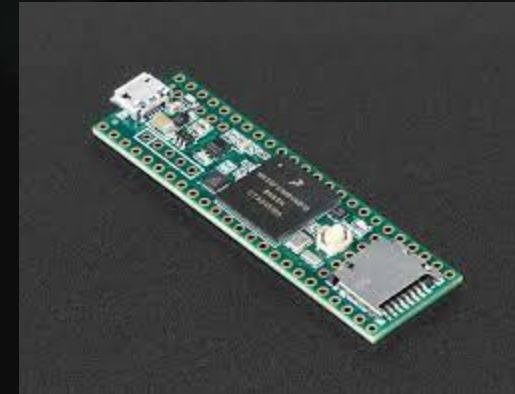
An embedded system is a specialized computer system designed to perform specific tasks within a larger system, combining both hardware and software components.

The hardware typically includes elements like a microcontroller or microprocessor, memory, input/output interfaces, and communication ports.

The software, often called firmware, is embedded in non-volatile memory and is programmed to meet specific constraints such as limited memory, power, and processing speed.

Embedded systems are used in a wide variety of applications, including consumer electronics (like microwaves), automotive systems, medical devices, and industrial automation systems.

About 2% of microprocessors are used in desktop computers, with 98% going to embedded systems.



➤ Embedded Systems

Smart Traffic Lights: Use sensors and timers to control traffic flow.

Digital Cameras: Contain microprocessors to process and store images.

Wearable Devices (e.g., Smartwatches, Fitness Trackers): Track health metrics and provide notifications.

Home Appliances (e.g., Refrigerators, Washing Machines): Use microcontrollers to control temperature, timers, and other functions.

Automotive Systems (e.g., Anti-lock Braking Systems, GPS Navigation): Use embedded systems to control various vehicle functions.

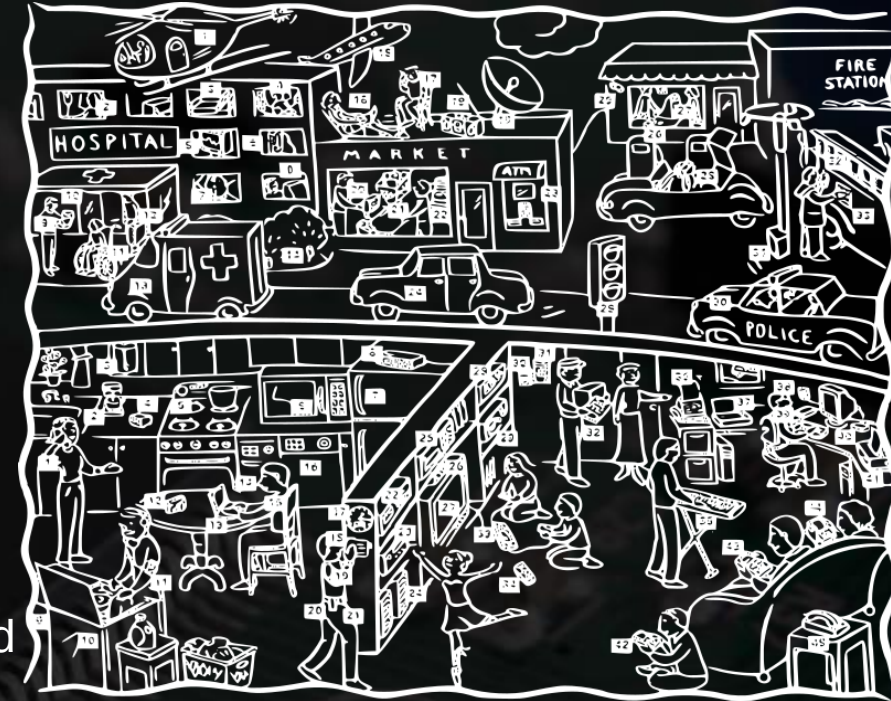
Medical Devices (e.g., Insulin Pumps, Portable Defibrillators): Use embedded systems to monitor and control medical conditions.

Gaming Consoles: Contain embedded systems to process graphics and gameplay.

Smart Home Devices (e.g., Thermostats, Security Systems): Use embedded systems to control and automate home functions.

Industrial Control Systems: Use embedded systems to monitor and control industrial processes.

Robotics: Use embedded systems to control and interact with the environment.



> Embedded Systems



➤ Key Components of Embedded Systems

Processor: This is the brain of the system, typically a microcontroller or microprocessor that executes instructions and processes data.

Memory: Embedded systems require RAM for temporary data storage and ROM/Flash memory to store the program (firmware).

Input/Output Interfaces: These are used to communicate with the external environment, which includes sensors (input) and actuators or displays (output).

Power Supply: Provides the necessary electrical power for the system to operate, often optimized for low power consumption.

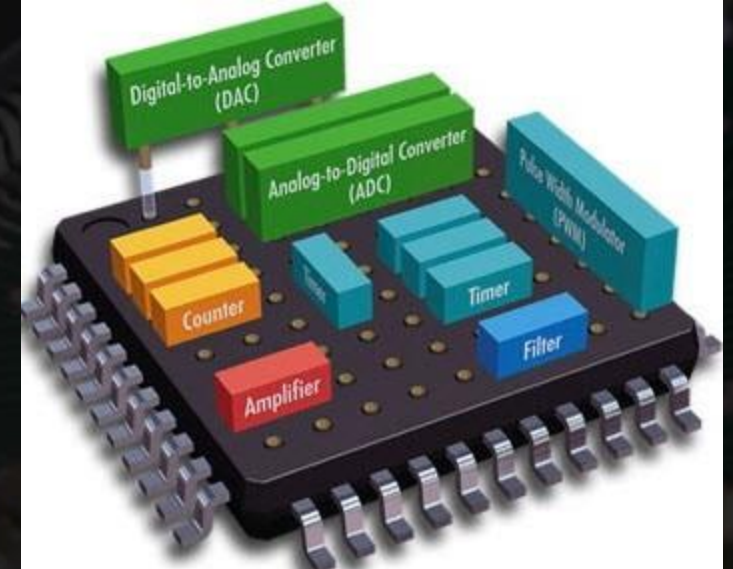
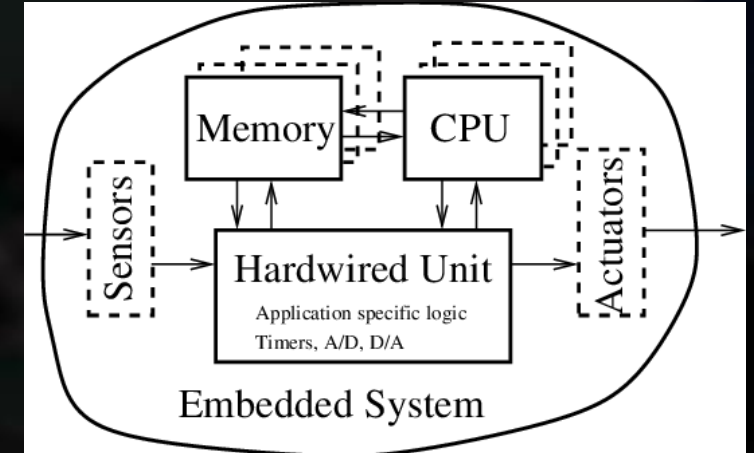


➤ Key Components of Embedded Systems

Timers: Essential for real-time tasks, timers manage time-critical operations by generating interrupts or keeping track of execution time.

Communication Ports: These enable data exchange with other systems or devices, commonly using protocols like UART, SPI, I2C, or CAN Bus.

Application-Specific Integrated Circuits (ASICs): These are customized hardware components designed for specific tasks to enhance performance and efficiency.



➤ Challenges in Embedded Systems

General-purpose computer applications are often written in high-level languages such as C++, Python, Java etc.

These languages abstract away many of the hardware-specific details, making the code portable across different hardware architectures (x86, ARM, etc.) and operating systems.

Tools like compilers, virtual machines, containerization, and adherence to standards like POSIX and web technologies further enhance portability, making it easier to run code on different platforms with minimal modification.

This flexibility is a key strength of general-purpose computer development, contrasting with the more tightly integrated and resource-constrained world of embedded systems.

Many embedded systems run bare-metal code (without an operating system) or use highly specialized real-time operating systems (RTOS) that are designed for specific tasks.

The lack of standardized OS APIs (like POSIX) means that code written for one RTOS or bare-metal environment is not easily transferrable to another.

The minimalistic nature of embedded systems, with limited CPU, memory, and power, often necessitates writing highly optimized code tailored to the specific hardware's limitations, reducing portability.

Embedded systems use a wide variety of processor architectures (e.g., ARM, AVR, PIC, RISC-V), each with unique instruction sets, memory management, and peripheral interfaces.

Code developed for one architecture is not inherently compatible with others. Embedded systems code is not portable because it is highly tailored to specific hardware, lacks abstraction layers like operating systems, uses low-level programming to directly control hardware peripherals, and often relies on proprietary toolchains and custom RTOS.

These constraints make it difficult to transfer code between different hardware platforms without significant rewriting or adaptation.

➤ Key differences between microcontrollers and microprocessors

Features	Microcontrollers	Microprocessors
Integrations	Microcontrollers integrate CPU, memory and I/O ports into a single chip.	Microprocessors require external components such as memory and additional peripherals.
Applications	Microcontrollers are better suited for specific, low-power or ultra-low power applications within embedded systems, such as home appliances like washing machines or IoT devices.	Microprocessors are more suitable for general-purpose and high-performance applications requiring more processing power, such as personal computers or the types of servers used in data centers.
Performance	Microcontrollers are optimized for efficiency and real-time processing and operate at lower clock speeds up to 200 MHz.	Microprocessors are built for more demanding and complicated computations and can operate at clock speeds above 1 GHz.
Cost	Microcontroller hardware is generally inexpensive and because they do not require specialized programming knowledge, they add very little to overall project budgets.	Microprocessors are better suited for specific, low-power or ultra-low-power applications within embedded systems, such as home appliances like washing machines or IoT devices.

➤ Key differences between embedded systems and general-purpose computers

	Embedded Systems	General-Purpose Computers
Purpose	Designed for a specific task (e.g., washing machines, medical devices).	Can perform multiple tasks (e.g., laptops, desktops).
Hardware	Compact, task-specific hardware with limited resources (microcontroller, sensors).	Powerful hardware (CPU, GPU) designed for flexibility and diverse applications.
Operating System	Often run on a real-time OS or bare metal.	Use full-featured OS (Windows, MacOS, Linux).
Power Consumption	Optimized for low power.	Higher power consumption due to broader capabilities.
User Interaction	Minimal or no direct interaction (e.g., control panels, sensors).	High interaction with keyboards, screens, etc.

➤ Conditions in Embedded Systems Development

Embedded systems often operate within strict resource limitations, including limited processing power, memory, and energy. Developers must optimize code and hardware to function efficiently under these constraints.

Many embedded systems need to respond to inputs in real-time. This requires precise timing and often the use of Real-Time Operating Systems (RTOS) to manage task execution and interrupt handling.

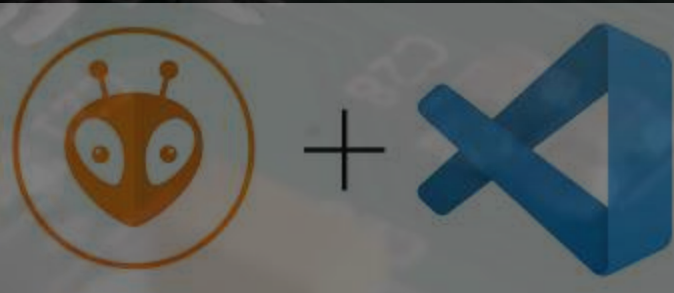
In critical applications, such as automotive systems or medical devices, embedded systems must adhere to stringent reliability and safety standards. Failures can have severe consequences, so systems are often subject to rigorous testing and validation.

➤ Development Tools & Environment

Firmware is software used to control a device's hardware. It is typically stored in ROM or flash memory and controls the various hardware components of the device, such as the processor, memory, and input/output peripherals.

Firmware development often involves writing code in a low-level programming language, such as C / C++, Python or assembly, and working closely with hardware engineers to design and implement the firmware for a device.

Firmware is usually specific to a particular device or platform and is often customized for that device. .



Safety

➤ Functional safety

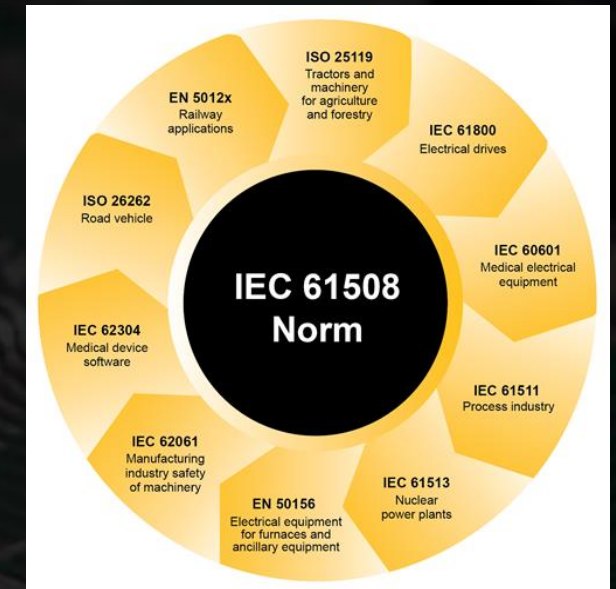
Definition and Examples: Safety-critical systems are those whose failure or malfunction can result in loss of life, injury, or damage to property or the environment. Examples include aircraft control systems, robotic surgery machines, railway signal control systems, and braking systems in vehicles.

Key Concepts Hazard: a condition that can cause harm or damage Risk: the probability of a system causing harm or damage in the case of a hazardous event Functional Safety: the absence of unreasonable risk of harming people or damaging the environment due to hazards caused by malfunctioning behaviour of systems Dependability: a general term that includes reliability, availability, and safety

➤ Functional safety

Development of Safety-Critical Systems Risk can never be completely eliminated, but probabilities can be reduced Development is performed according to the state of the art in all aspects of safety The burden of proof is with the manufacturer, and documented proof must be produced International standards are required, such as IEC 61508, IEC 62304, and ISO 26262.

Types of Safety Critical Systems Fail-operational systems: can complete the operation even if part of the system fails Fail-safe systems: switch to a safe mode and inform an operator Fail-secure systems: become secure and lock up to minimize harm Fail-passive systems: become passive and hand over control to an operator Fault-tolerant systems: continue to operate even if part of the system fails.



➤ Functional safety

Development of Safety-Critical Software Systems Uses the V-Model framework at system, hardware, and software levels Standards like ISO 26262, AUTOSAR, and MISRA C are used Development involves safety requirements specification, architectural design, unit design and implementation, unit testing, integration testing, system testing, and verification and validation of safety requirements.

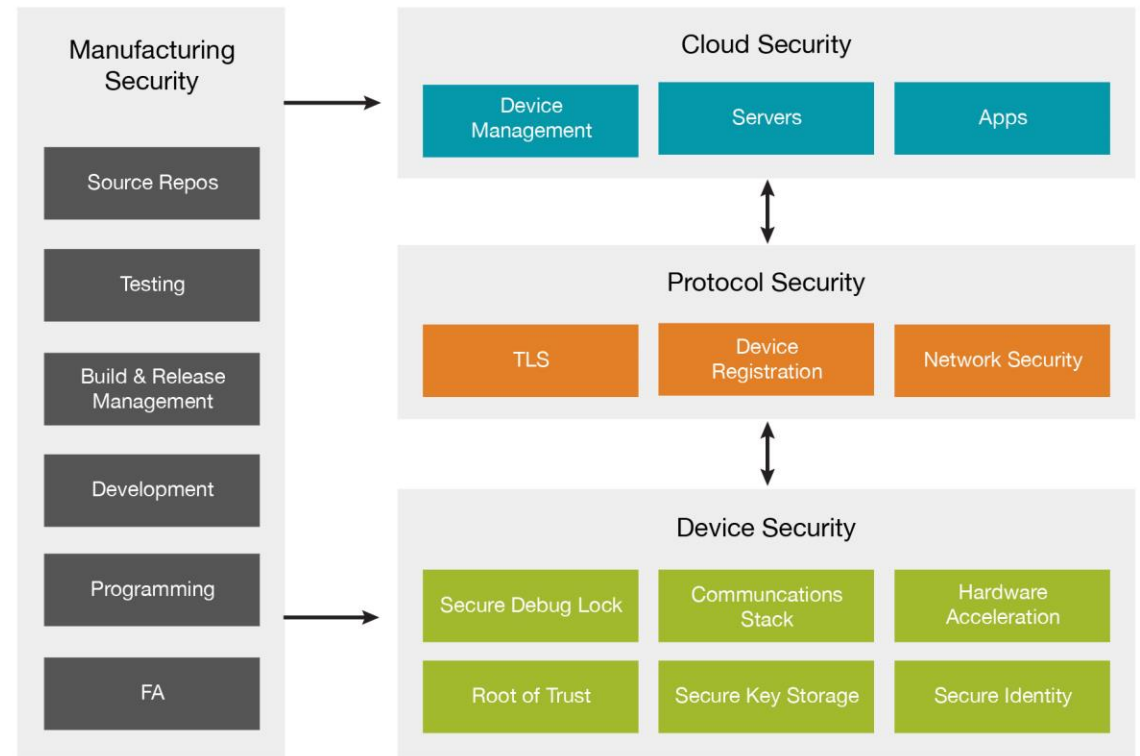
Guidelines for Safety-Critical Software Systems Modelling and coding guidelines, such as enforcement of low complexity and use of language subsets Principles for software architectural design, such as hierarchical structure, restricted size and complexity, and loose coupling between components.

Security

➤ Embedded Security

Security is protection against external threats which can cause harm or damage.

Cyber security is the practice of defending computers, servers, mobile devices, electronic systems, networks, and data from malicious attacks. It's also known as information technology security or electronic information security.



➤ Embedded Security

Implementing security policies in embedded systems requires addressing both hardware and software level challenges, which can be challenging due to various factors.

Irregular software update:

Most embedded systems are not regularly updated for security updates, allowing them to function without human intervention. To ensure security, self-updating programs should be employed to automatically update these systems with necessary software updates.

Attack reproductivity:

as the embedded systems are produced in mass production their interior structure and properties are same. If any attacker gets successful in attacking one device, he can attack other identical devices as well.

Rules and regulations:

There are no fixed rules and regulations exist for ensuring the security of these systems. Therefore, sometimes for the commercial purposes the industries compromise its security with the set rules and regulations laid by the authority.

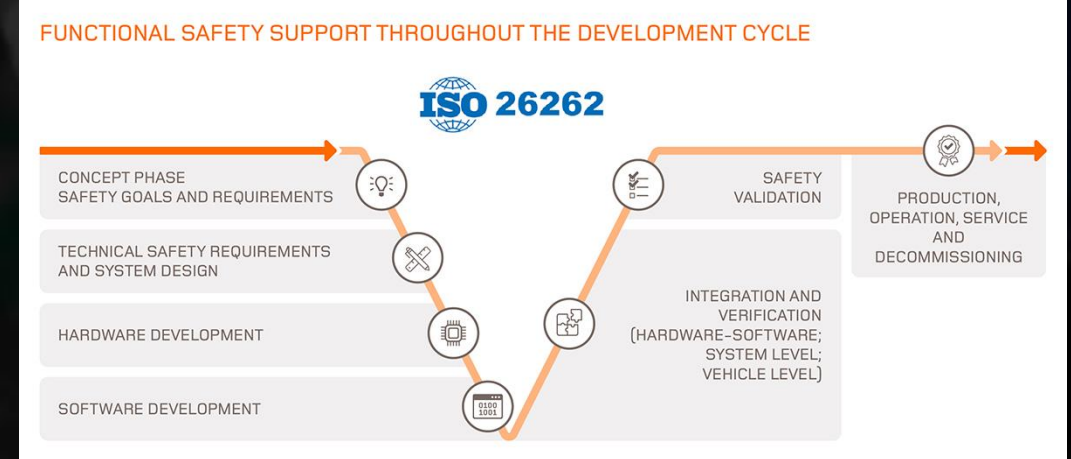
Safety Vs Security

➤ Safety Vs Security

Safety is absence of internal failures which can cause harm or damage • Define all points: FMEA, safety features, coding roles and after implementation we have to test it.

Security is protection against external threats which can cause harm or damage

Cyber security is the practice of defending computers, servers, mobile devices, electronic systems, networks, and data from malicious attacks. It's also known as information technology security or electronic information security.



> HARA and TARA

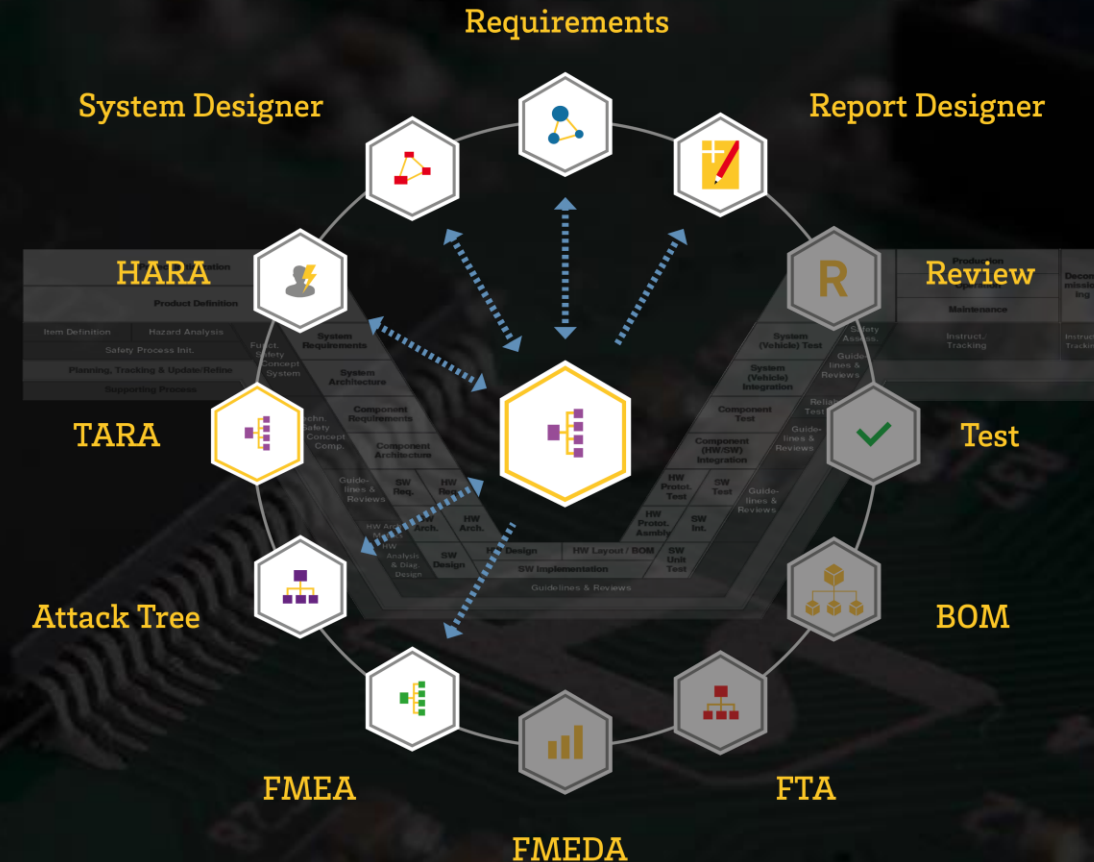
When we are talking about Safety or Security, we have to consider two important points:

HARA: Hazard Analysis and Risk Assessment

Defines how the risk posed by electronics in road vehicles should be estimated and assessed. In particular, it is about the risk of electrical, electronic and software systems malfunctioning.

TARA: Threat Assessment and Risk Assessment.

The systematic risk assessment within the concept phase of an automotive development project.



➤ REFERENCES

- [The Apollo Guidance Computer \(AGC\)](#)
- [Functional safety in embedded systems](#)
- [Embedded Systems Security](#)